

Függvények C++-ban: paraméter és return

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ez a jegyzet úgy épül fel, hogy egyedül is végig tudj menni rajta. Három dolog segít ebben:

- **Kis lépések.** Minden újdonságot előbb egy kész kódon próbálsz ki, és csak utána írsz sajátot.
- **Ellenőrzés.** A feladatok megadják, mit kell kapnod. Ha a programod ugyanazt írja ki, jó úton jársz, és mehetsz tovább.
- **Hibakereső.** Ha hibaüzenetet kapsz, lapozz a jegyzet végére. A Hibakereső táblázat megmondja, mit jelent az üzenet, és mit tegyél.

A jelölések a szokásosak: a **Próbáld ki** kódjait gépeld be és futtasd, a **Tippelj** részeknél előbb írd le a tippedet, a **Feladat** pedig a te munkád. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Ha elakadsz, ebben a sorrendben próbálkozz:

1. Olvasd el újra a feladat előtti mintakódot. A megoldás szinte mindig annak az átalakítása.
2. Nézd meg a Hibakeresőt a jegyzet végén.
3. Kérdezd meg a padtársadat.
4. Ha ez sem segített, szólj a tanárodnak.

1. Az első saját függvény

A függvény egy névvel ellátott programrészlet. Egyszer megírod, és ahányszor kell, a nevével meghívod.

Próbáld ki - ismétlődő kód

```
#include <iostream>

int main() {
    std::cout << "-----" << std::endl;
    std::cout << "Informatika szakkor" << std::endl;
    std::cout << "-----" << std::endl;
    std::cout << "Mai tema: függvények" << std::endl;
    std::cout << "-----" << std::endl;
    return 0;
}
```

Futtasd, aztán cseréld le mindhárom vonalban a kötőjeleket egyenlőségjelekre. Hány sort kellett átírnod?

Próbáld ki - ugyanez függvénnyel

```
#include <iostream>

void vonal() {
    std::cout << "-----" << std::endl;
}

int main() {
    vonal();
    std::cout << "Informatika szakkor" << std::endl;
    vonal();
    std::cout << "Mai tema: fuggvenyek" << std::endl;
    vonal();
    return 0;
}
```

Ugyanazt írja ki, mint az előző program. Most is cseréld le a kötőjeleket egyenlőségjelekre. Hány sort kellett átírnod?

Nézzük meg a függvény részeit:

- **void** – ez a függvény nem ad vissza eredményt, csak elvégez valamit. (A 3. fejezetben lesz másféle is.)
- **von**al – a függvény neve. Te választod, mint egy változót.
- **()** – ide kerülnek majd a paraméterek. Most üres.
- **{ ... }** – a függvény törzse: ezek az utasítások futnak le.
- **von**al(); – a **hívás**. Ennél a sornál hajtja végre a program a függvény törzsét, aztán a következő sorral folytatja.

Észrevetted? A **main** is egy függvény. A program mindig a **main** elején indul.

Tippelj - milyen sorrendben fut?

Írd le a füzetedbe, milyen betűk jelennek meg, és milyen sorrendben. Csak utána futtasd!

```
#include <iostream>

void elso() {
    std::cout << "A" << std::endl;
}

void masodik() {
    std::cout << "B" << std::endl;
}

int main() {
    std::cout << "C" << std::endl;
    masodik();
    elso();
    masodik();
    return 0;
}
```

A függvények abban a sorrendben futnak le, ahogy meghívod őket – nem abban, ahogy megírtad.

Kísérlet - függvény hívás nélkül

Töröld ki az előző programból az **elso();** sort. Megjelenik még az **A** betű? Fogalmazd meg egy mondatban a szabályt.

Kísérlet - hol legyen a függvény?

Vágd ki az `első` függvényt (mind a három sorát), és illeszd be a program legvégére, a `main` záró kapcsos zárójele után. Lefordul a program? Olvasd el a hibaüzenetet, és keresd meg a Hibakeresőben.

Figyelem! A saját függvényeidet mindig a `main` elé írd. A fordító felülről lefelé olvas: amit még nem látott, azt nem ismeri.

Feladatok

1.1. feladat ★☆☆ - Üdvözlés

Fejezd be ezt a programot. Az `udvozles` függvény két sort írjon ki (`Szia!` és `Jo munkat!`), a `main` pedig hívja meg háromszor.

```
#include <iostream>

void udvozles() {
    std::cout << "Szia!" << std::endl;
    // ide írd a második kiírást
}

int main() {
    // itt hívd meg háromszor
    return 0;
}
```

Ellenőrzés: hat sornak kell megjelennie, felváltva `Szia!` és `Jo munkat!`.

1.2. feladat ★☆☆ - Csillagsor

Írj `csillagsor` nevű függvényt, amely ciklussal tíz csillagot ír ki egy sorba. Hívd meg kétszer, és a két hívás között írd ki a nevedet.

Ellenőrzés:

```
*****
Anna
*****
```

1.3. feladat ★☆☆ - Három függvény

Írj három függvényt: `eleje`, `kozepe`, `vege`. Mindegyik egy sort ír ki: `Egyszer volt, - hol nem volt, - volt egyszer egy program`. A `main` hívja meg őket úgy, hogy a mese jó sorrendben jelenjen meg.

Ellenőrzés: három sor, a fenti sorrendben. Ezután cseréld fel a hívásokat a `main`-ben. A függvényekhez hozzá kellett nyúlnod?

1.4. feladat ★☆☆ - Függvény a függvényben

Írj `fejlec` nevű függvényt, amely meghívja a `csillagsor` függvényt, kiírja az `INFORMATIKA` szót, majd újra meghívja a `csillagsor`-t. A `main`-ben most csak egyetlen sor legyen: a `fejlec();` hívás.

Ellenőrzés: ugyanaz a három sor, mint az 1.2. feladatban, csak a név helyén `INFORMATIKA` áll. Gondold végig: melyik függvénynek kell előbb szerepelnie a fájlban, és miért?

2. Paraméterek: adat a függvénynek

A paraméterrel adatot adsz át a függvénynek. Így ugyanaz a függvény minden hívásnál mást csinálhat.

Próbáld ki

```
#include <iostream>

void vonal(int hossz) {
    for (int i = 0; i < hossz; i++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

int main() {
    vonal(5);
    vonal(20);

    int n = 12;
    vonal(n);
    vonal(n / 2);
    return 0;
}
```

A zárójelben álló `int hossz` a **paraméter**: egy változó, amely a híváskor kapja meg az értékét. A `vonala(5)` hívásnál a `hossz` értéke 5, a `vonala(20)` hívásnál 20.

Híváskor a zárójelbe szám, változó vagy számítás is kerülhet. Milyen hosszú vonalat rajzolt az utolsó hívás?

Próbáld ki - több paraméter

Cseréld le a `vonala` függvényt erre a változatra:

```
void vonal(int hossz, char jel) {
    for (int i = 0; i < hossz; i++) {
        std::cout << jel;
    }
    std::cout << std::endl;
}
```

A `main`-ben hívd így: `vonala(10, '*');` és `vonala(3, '#');`

A paramétereket vessző választja el, és mindegyiknek külön meg kell adni a típusát.

Tippelj - számít a sorrend?

```
#include <iostream>

void kivon(int a, int b) {
    std::cout << a - b << std::endl;
}

int main() {
    kivon(10, 3);
    kivon(3, 10);

    int a = 1;
    int b = 8;
    kivon(b, a);
    return 0;
}
```

A harmadik hívás a trükkös. Az értékek a **helyük** szerint kerülnek a paraméterekbe, nem a nevük szerint.

Próbáld ki - sztring paraméter

```
#include <iostream>
#include <string>

void koszon(std::string nev) {
    std::cout << "Szia, " << nev << "!" << std::endl;
}

int main() {
    koszon("Anna");

    std::string valaki;
    std::cin >> valaki;
    koszon(valaki);
    return 0;
}
```

Kísérlet - hibás hívások

Írd be az alábbi sorokat **egyessel** a kétparaméteres **von** függvényt használó programod **main**-jébe. Olvasd el a hibaüzenetet, keresd meg a Hibakeresőben, aztán töröld a hibás sort.

```
von();
von(5, '*', 3);
von("ot", '*');
```

Feladatok

2.1. feladat ★☆☆ - Köszönés névvel

Gépelj be a **koszon** függvényt, és hívd meg három különböző névvel.

Ellenőrzés: három sor, például **Szia, Bence!**

2.2. feladat ★☆☆ - Számsor

Írj **szamsor(int n)** függvényt, amely kiírja a számokat 1-től **n**-ig egy sorba, szóközzel elválasztva.

Ellenőrzés: a **szamsor(5)**; hívás kimenete **1 2 3 4 5**, a **szamsor(1)**; hívásé **1**.

2.3. feladat ★☆☆ - Szorzótábla

Írj **szorzotabla(int n)** függvényt, amely kiírja az **n** szorzótábláját 1-től 10-ig.

Ellenőrzés: a **szorzotabla(7)**; hívás első sora **1 x 7 = 7**, az utolsó **10 x 7 = 70**.

2.4. feladat ★☆☆ - Téglalap

Írj **teglalap(int szelesseg, int magassag)** függvényt, amely csillagokból téglalapot rajzol. **Tipp:** a függvényed meghívhatja a kétparaméteres **von** függvényt.

Ellenőrzés: a **teglalap(5, 3)**; hívás kimenete:

```
*****
*****
*****
```

2.5. feladat ★★☆☆ - Háromszög

Írj `haromszog(int n)` függvényt: az első sorban egy, a másodikban kettő, az utolsóban `n` csillag áll.

Ellenőrzés: a `haromszog(4)`; hívás kimenete:

```
*
**
***
****
```

2.6. feladat ★★☆☆ - Bemutató

Írj `bemutat(std::string nev, int kor)` függvényt.

Ellenőrzés: a `bemutat("Anna", 17)`; hívás kimenete `Anna 17 éves, jövőre 18 lesz.`

3. Visszatérési érték: a függvény válaszol

A `return` utasítással a függvény eredményt ad vissza oda, ahonnan meghívták.

Próbáld ki

```
#include <iostream>

int negyzet(int x) {
    return x * x;
}

int main() {
    std::cout << negyzet(5) << std::endl;

    int eredmeny = negyzet(7);
    std::cout << eredmeny << std::endl;
    return 0;
}
```

Mi változott az eddigiekhez képest?

- A név előtt `void` helyett `int` áll: a függvény egész számot ad vissza.
- A `return` után áll az eredmény.
- A hívás helyére az eredmény kerül. Képzeld úgy, hogy a programban a `negyzet(5)` helyén a 25 áll.

Tippelj

Az eredménnyel bármit megtehetsz, amit egy számmal. Mit ír ki ez a részlet?

```
std::cout << negyzet(3) + negyzet(4) << std::endl;
std::cout << negyzet(negyzet(2)) << std::endl;

int a = 6;
if (negyzet(a) > 30) {
    std::cout << "nagy" << std::endl;
}
```

Kísérlet - kiírni vagy visszaadni?

Ez a fejezet legfontosabb kísérlete. Legyen a programodban mindkét függvény:

```
void kiirNegyzet(int x) {
    std::cout << x * x << std::endl;
}

int negyzet(int x) {
    return x * x;
}
```

Írd be ezeket a sorokat **egyesével** a **main**-be, és figyeld meg, mi történik:

```
kiirNegyzet(5);
negyzet(5);
int a = negyzet(5) + 1;
int b = kiirNegyzet(5);
```

Melyik sor írt ki valamit? Melyik után nem történt semmi látható? Melyik hibás?

Figyelem! A kiírás és a visszaadás nem ugyanaz! Amit a függvény kiír, az megjelenik a képernyőn, de a program nem tud vele tovább számolni. Amit visszaad, azzal a hívó azt tesz, amit akar: kiírja, eltárolja vagy számol vele.

Tippelj - mi történik a return után?

```
int abszolot(int x) {
    if (x < 0) {
        return -x;
    }
    return x;
}
```

Kövesd végig papíron az **abszolot(-4)** és az **abszolot(9)** hívást. Melyik **return** fut le az egyik, és melyik a másik esetben? Utána írasd ki mindkettőt.

A **return** azonnal befejezi a függvényt. Ami utána áll, az már nem fut le.

Kísérlet - ha elmarad a return

Töröld ki a **negyzet** függvényből a **return x * x;** sort, és írasd ki a **negyzet(5)** értékét. Mit kapsz? Futtasd többször is.

Figyelem! Ha a függvény típusa nem **void**, akkor minden lehetséges úton **return**-nel kell végződnie. A fordító a hiányzó **return** miatt legfeljebb figyelmeztet, a program pedig értelmetlen számot ad, vagy összeomlik.

Feladatok

3.1. feladat ★☆☆ - Duplázó

Írj **int duplaz(int x)** függvényt, amely visszaadja a szám kétszeresét. A **main**-ben írasd ki az eredményt.

Ellenőrzés: **duplaz(4)** → 8, **duplaz(-3)** → -6, **duplaz(0)** → 0.

3.2. feladat ★☆☆ - Összeg

Írj **int osszeg(int a, int b)** függvényt.

Ellenőrzés: **osszeg(3, 4)** → 7, **osszeg(-5, 5)** → 0.

3.3. feladat ★☆☆ - Téglalap adatai

Írj két függvényt: az `int terület(int a, int b)` a téglalap területét, az `int kerulet(int a, int b)` a kerületét adja vissza.

Ellenőrzés: `terület(3, 5) → 15`, `kerulet(3, 5) → 16`.

3.4. feladat ★☆☆ - Nagyobb

Írj `int nagyobb(int a, int b)` függvényt, amely a két szám közül a nagyobbat adja vissza.

Ellenőrzés: `nagyobb(3, 7) → 7`, `nagyobb(7, 3) → 7`, `nagyobb(5, 5) → 5`.

3.5. feladat ★☆☆ - Legnagyobb a három közül

Írj `int legnagyobb(int a, int b, int c)` függvényt. Ne írd új `if`-eket: használd fel a `nagyobb` függvényt!

Ellenőrzés: `legnagyobb(2, 9, 4) → 9`, `legnagyobb(9, 2, 4) → 9`, `legnagyobb(2, 4, 9) → 9`.

3.6. feladat ★☆☆ - Átlag

Írj `double atlag(int a, int b)` függvényt, amely két egész szám átlagát adja vissza.

Ellenőrzés: `atlag(3, 4) → 3.5`, `atlag(10, 20) → 15`. Ha az első esetben 3-at kaptál, emlékezz az egész osztás csapdájára a vektoros jegyzetből.

3.7. feladat ★☆☆ - Hőmérséklet

Írj `double fahrenheit(double celsius)` függvényt. A képlet: $F = C \cdot 9 / 5 + 32$.

Ellenőrzés: `fahrenheit(0) → 32`, `fahrenheit(100) → 212`, `fahrenheit(37) → 98.6`.

3.8. feladat ★☆☆ - Faktoriális

Írj `int faktorialis(int n)` függvényt. Az `n` faktoriálisa a számok szorzata 1-től `n`-ig.

Ellenőrzés: `faktorialis(5) → 120`, `faktorialis(1) → 1`, `faktorialis(10) → 3628800`.

4. Igaz vagy hamis? Logikai függvények

Egy `bool` típusú függvény egy kérdésre válaszol igennel vagy nemmel. Ettől a feltételeid olvashatóbbak lesznek.

Próbáld ki

```
#include <iostream>

bool paros(int x) {
    return x % 2 == 0;
}

int main() {
    int n;
    std::cin >> n;

    if (paros(n)) {
        std::cout << "paros" << std::endl;
    } else {
        std::cout << "paratlan" << std::endl;
    }
    return 0;
}
```

Az `x % 2 == 0` kifejezés értéke maga igaz vagy hamis. A függvény ezt adja vissza.

Olvasd fel hangosan az `if` sorát: „ha páros `n`”. Ezért érdemes a logikai függvénynek kérdésre válaszoló nevet adni.

Kísérlet - mit ír ki a bool?

```
std::cout << paros(4) << " " << paros(7) << std::endl;
```

A C++ az igazat 1-nek, a hamisat 0-nak írja ki. Az ellenőrzéseknél ezért 1-et és 0-t fogsz látni.

Próbáld ki - keresés függvényben

Ez a függvény azt dönti el, van-e a számnak 1-en és önmagán kívül más osztója:

```
bool osszetett(int n) {  
    for (int i = 2; i < n; i++) {  
        if (n % i == 0) {  
            return true;  
        }  
    }  
    return false;  
}
```

Kövesd végig papíron $n = 9$ és $n = 7$ esetén. Hányadik körben áll meg a függvény az egyik, és hányadikban a másik esetben? Utána írasd ki: `osszetett(9)`, `osszetett(7)`, `osszetett(15)`.

Figyeld meg a szerkezetet: amint találtunk egy osztót, a `return true` azonnal kilép. A `return false` a ciklus **után** áll – oda csak akkor jutunk el, ha egyetlen osztót sem találtunk. Ugyanez volt a szabály a vektoros keresésnél is.

Feladatok

4.1. feladat ★☆☆ - Pozitív-e?

Írj `bool pozitiv(int x)` függvényt.

Ellenőrzés: `pozitiv(5) → 1`, `pozitiv(-2) → 0`, `pozitiv(0) → 0`.

4.2. feladat ★☆☆ - Osztható-e?

Írj `bool oszthato(int a, int b)` függvényt, amely megmondja, osztható-e `a` a `b` számmal.

Ellenőrzés: `oszthato(12, 4) → 1`, `oszthato(12, 5) → 0`.

4.3. feladat ★☆☆ - Nagykorú

Írj `bool nagykoru(int kor)` függvényt. A `main`-ben olvass be egy életkort, és a függvény segítségével írd ki, hogy az illető nagykorú-e.

Ellenőrzés: `nagykoru(18) → 1`, `nagykoru(17) → 0`.

4.4. feladat ★☆☆ - Határok között

Írj `bool kozte(int x, int also, int felso)` függvényt: igaz, ha `x` az `also` és a `felso` közé esik. A határok is beleszámítanak.

Ellenőrzés: `kozte(5, 1, 10) → 1`, `kozte(10, 1, 10) → 1`, `kozte(11, 1, 10) → 0`.

4.5. feladat ★☆☆ - Magánhangzó

Írj `bool maganhangzo(char c)` függvényt (a, e, i, o, u). Használd fel arra, hogy megszámold egy beolvasott szó magánhangzóit.

Ellenőrzés: `maganhangzo('a') → 1`, `maganhangzo('k') → 0`. Az `informatika` szóban 5 magánhangzó van.

4.6. feladat ★☆☆ - Szökőév

Írj `bool szokoev(int ev)` függvényt. A szabály: szökőév, ha osztható 4-gyel; de ha 100-zal is osztható, akkor csak abban az esetben, ha 400-zal is.

Ellenőrzés: `szokoev(2024) → 1`, `szokoev(2023) → 0`, `szokoev(1900) → 0`, `szokoev(2000) → 1`.

4.7. feladat ★★★ - Prímszám

Írj `bool prim(int n)` függvényt. **Tipp:** az `osszetett` függvény mintájára dolgozz. Vigyázz: az 1 nem prím.

Ellenőrzés: `prim(2), prim(3), prim(17) → 1; prim(1), prim(9), prim(15) → 0.`

5. Ami a függvényben történik, az ott is marad

A függvény változói csak a függvényen belül léteznek. A paraméter pedig **másolatot** kap az átadott értékről.

Kísérlet - ki látja a változót?

```
#include <iostream>

void kiir() {
    std::cout << szam << std::endl;
}

int main() {
    int szam = 5;
    kiir();
    return 0;
}
```

Lefordul? Olvasd el a hibaüzenetet.

A `szam` a `main` változója, ezért a `kiir` nem látja. Minden függvény csak a saját változóit és a paramétereit ismeri. Ezeket **helyi** (lokális) változóknak hívjuk.

Tippelj - a paraméter másolat

```
#include <iostream>

void novel(int x) {
    x = x + 1;
    std::cout << "A fuggvényben: " << x << std::endl;
}

int main() {
    int a = 5;
    novel(a);
    std::cout << "A mainben: " << a << std::endl;
    return 0;
}
```

Mit ír ki a két sor? A második meglepő lehet: a függvény csak a saját másolatát növelte meg, a `main` változója nem változott.

Próbáld ki - két megoldás

Első megoldás: a függvény visszaadja az új értéket, és a `main` elteszi.

```
int novel(int x) {
    return x + 1;
}
```

A `main`-ben így hívd: `a = novel(a);`

Második megoldás: a paraméter típusa után `&` jelet írsz. Ekkor a függvény nem másolatot kap, hanem magát az eredeti változót.

```
void novel(int& x) {  
    x = x + 1;  
}
```

A `main`-ben a hívás marad `novel(a);`. Próbáld ki mindkettőt!

Tipp: Számolásnál az első megoldást válaszd (`return`). A `&` jelet akkor használd, ha a függvénynek egyszerre több változót, vagy egy egész vektort kell módosítania.

Tippelj - két egyforma nevű változó

```
#include <iostream>  
  
void szamol() {  
    int i = 100;  
    std::cout << i << std::endl;  
}  
  
int main() {  
    int i = 1;  
    szamol();  
    std::cout << i << std::endl;  
    return 0;  
}
```

A két `i` két külön változó, csak a nevük egyezik. Ezért nem kell attól félned, hogy egy függvény „elrontja” a másik változóit.

Feladatok

5.1. feladat ★☆☆ - Javítás

Javítsd ki a fejezet első programját úgy, hogy a `kiir` függvény paraméterként kapja meg a kiírandó számot.

Ellenőrzés: a program kimenete `5`.

5.2. feladat ★☆☆ - Mennyi lesz?

Előbb papíron dönts el, mit ír ki ez a program, aztán futtasd.

```
#include <iostream>  
  
void felez(int x) {  
    x = x / 2;  
}  
  
int main() {  
    int a = 40;  
    felez(a);  
    felez(a);  
    std::cout << a << std::endl;  
    return 0;  
}
```

Ezután javítsd ki `return` használatával úgy, hogy a két felezés tényleg megtörténjen.

Ellenőrzés: a javított program kimenete `10`.

5.3. feladat ★★☆☆ - Csere

Írj `void csere(int& a, int& b)` függvényt, amely megcseréli két változó értékét.

Ellenőrzés: ha `x = 3` és `y = 8`, akkor a `csere(x, y);` hívás után `x` értéke 8, `y` értéke 3. Próbáld ki a két `&` jel nélkül is: mi történik?

5.4. feladat ★★☆☆ - Biztonságos beolvasás

Írj `int beker(int also, int felso)` függvényt. Addig kérjen be számot a felhasználtól, amíg az az `also` és a `felso` közé nem esik (a határok megengedettek). A jó számot adja vissza.

Ellenőrzés: `int jegy = beker(1, 5);` hívásnál írd be sorban: 9, 0, 3. A `jegy` értéke 3 lesz.

6. Függvények vektorral és sztringgel

Egy függvény paramétere vektor vagy sztring is lehet, és ilyet vissza is adhat.

Próbáld ki - vektor paraméter

```
#include <iostream>
#include <vector>

int osszeg(std::vector<int> v) {
    int s = 0;
    for (int i = 0; i < v.size(); i++) {
        s = s + v[i];
    }
    return s;
}

int main() {
    std::vector<int> jegyek = {5, 3, 4, 5, 2};
    std::vector<int> pontok = {42, 67, 55};

    std::cout << osszeg(jegyek) << std::endl;
    std::cout << osszeg(pontok) << std::endl;
    return 0;
}
```

Az összegzést egyszer írtad meg, mégis két különböző vektorra használod. A vektoros jegyzet minden mintája (megszámlálás, legnagyobb elem, keresés) megírható így, függvényként.

Próbáld ki - sztringet kap, sztringet ad vissza

```
#include <iostream>
#include <string>

std::string fordit(std::string s) {
    std::string uj = "";
    for (int i = 0; i < s.length(); i++) {
        uj = s[i] + uj;
    }
    return uj;
}

int main() {
    std::cout << fordit("alma") << std::endl;

    std::string szo = "informatika";
    std::string vissza = fordit(szo);
    std::cout << szo << " - " << vissza << std::endl;
    return 0;
}
```

Tippelj - megváltozik-e a vektor?

```
#include <iostream>
#include <vector>

void duplaz(std::vector<int> v) {
    for (int i = 0; i < v.size(); i++) {
        v[i] = v[i] * 2;
    }
}

int main() {
    std::vector<int> szamok = {1, 2, 3};
    duplaz(szamok);
    std::cout << szamok[0] << std::endl;
    return 0;
}
```

Mit ír ki? Gondolj az 5. fejezetre. Utána írd át a paramétert erre: `std::vector<int>& v`. Most mit ír ki?

Feladatok

6.1. feladat ★☆☆ - Kiíró

Írj `void kiir(std::vector<int> v)` függvényt, amely a vektor elemeit egy sorba írja ki, szóközzel elválasztva. A fejezet többi feladatában ezt használd a vektorok kiírására.

Ellenőrzés: az `{5, 3, 4}` vektorra a kimenet `5 3 4`.

6.2. feladat ★☆☆ - Ötösök

Írj `int hanyOtos(std::vector<int> v)` függvényt, amely megszámolja az ötösöket.

Ellenőrzés: az `{5, 3, 4, 5, 2}` vektorra az eredmény 2.

6.3. feladat ★☆☆ - Átlag

Írj `double atlag(std::vector<int> v)` függvényt. Az összeget ne számold ki újra: hívd meg az `osszeg` függvényt.

Ellenőrzés: az `{5, 3, 4, 5, 2}` vektorra az eredmény 3.8.

6.4. feladat ★★☆☆ - Legnagyobb

Írj `int legnagyobb(std::vector<int> v)` függvényt.

Ellenőrzés: a {42, 67, 55, 91, 38} vektorra 91, a {-5, -2, -9} vektorra -2.

6.5. feladat ★★☆☆ - Van-e benne?

Írj `bool vanBenne(std::vector<int> v, int keresett)` függvényt. **Tipp:** a 4. fejezet `osszetett` függvényének a szerkezetét kövesd.

Ellenőrzés: a {4, 8, 15} vektorban a 8-ra 1, a 7-re 0 az eredmény.

6.6. feladat ★★☆☆ - Betűszámláló

Írj `int hanyszor(std::string s, char c)` függvényt, amely megszámolja, hányszor szerepel a `c` karakter az `s` szövegben.

Ellenőrzés: `hanszor("informatika", 'a') → 2`, `hanszor("informatika", 'i') → 2`, `hanszor("informatika", 'z') → 0`.

6.7. feladat ★★☆☆ - Palindrom

Írj `bool palindrom(std::string s)` függvényt. Használd fel a `fordit` függvényt!

Ellenőrzés: `palindrom("kajak") → 1`, `palindrom("alma") → 0`.

6.8. feladat ★★☆☆ - Nagybetűs

Írj `std::string nagybetus(std::string s)` függvényt, amely a szöveg nagybetűs változatát adja vissza.

Ellenőrzés: `nagybetus("alma") → ALMA`.

6.9. feladat ★★☆☆ - Párosak

Írj `std::vector<int> parosak(std::vector<int> v)` függvényt, amely egy új vektorban adja vissza a páros elemeket.

Ellenőrzés: a {3, 8, 12, 7, 10} vektorra az eredmény (a `kiir` függvénnyel kiírva) 8 12 10.

6.10. feladat ★★☆☆ - Mindet növel

Írj `void novelMind(std::vector<int>& v, int mennyivel)` függvényt, amely a vektor minden elemét megnöveli.

Ellenőrzés: az {1, 2, 3} vektor a `novelMind(szamok, 10)`; hívás után 11 12 13.

7. Program függvényekből

Egy nagyobb programot úgy érdemes megírni, hogy minden részfeladat külön függvény. A `main` ezeket csak sorban meghívja.

Próbáld ki

```
#include <iostream>
#include <vector>

std::vector<int> beolvas(int db) {
    std::vector<int> v;
    for (int i = 0; i < db; i++) {
        int x;
        std::cin >> x;
        v.push_back(x);
    }
    return v;
}

int osszeg(std::vector<int> v) {
    int s = 0;
    for (int i = 0; i < v.size(); i++) {
        s = s + v[i];
    }
    return s;
}

double atlag(std::vector<int> v) {
    double s = osszeg(v);
    return s / v.size();
}

int main() {
    std::cout << "Hany jegy lesz? ";
    int db;
    std::cin >> db;

    std::vector<int> jegyek = beolvas(db);

    std::cout << "Osszeg: " << osszeg(jegyek) << std::endl;
    std::cout << "Atlag: " << atlag(jegyek) << std::endl;
    return 0;
}
```

Olvasd el csak a `main` függvényt. Érted, mit csinál a program, anélkül hogy a többi függvény belsejét megnéznéd? Ez a jó program ismertetőjele.

Három jó tanács a saját programjaidhoz:

- Egy függvény egyetlen dolgot csináljon, és a neve mondja meg, mit.
- A számoló függvény adja vissza az eredményt, ne írja ki. A kiírás a `main` dolga.
- Ne írd meg egyszerre az egész programot. Írj meg egy függvényt, próbáld ki a `main`-ból, és csak utána írd a következőt.

Feladatok

7.1. feladat ★★☆☆ - Bővítés

Bővítsd a fenti programot két függvénnyel: a `legnagyobb` a legjobb jegyet adja vissza, a `hanyJobb(std::vector<int> v, double hatar)` pedig azt, hány elem nagyobb a határnál. Írd ki a legjobb jegyet és azt, hány jegy jobb az átlagnál.

Ellenőrzés: ha a jegyek 5, 3, 4, 5, 2, akkor az összeg 19, az átlag 3.8, a legjobb jegy 5, és 3 jegy jobb az átlagnál.

7.2. feladat ★★☆☆ - Számológép

Írj négy függvényt: `osszead`, `kivon`, `szoroz`, `oszt`. Mindegyik két `double` számot kap, és az eredményt adja vissza. A `main` olvasson be egy műveletet ebben a formában: `8 / 2`, és a műveleti jel alapján hívja meg a megfelelő függvényt. **Tipp:** `std::cin >> a >> jel >> b;`, ahol a `jel` típusa `char`.

Ellenőrzés: `8 / 2` → 4, `3 * 5` → 15, `2.5 + 1` → 3.5, `1 - 4` → -3.

7.3. feladat ★★☆☆ - Jelszóellenőr függvényekkel

Írj három függvényt: `bool vanKisbetu(std::string s)`, `bool vanNagybetu(std::string s)`, `bool vanSzamjegy(std::string s)`. Írj egy negyediket is: az `eros` függvény akkor ad igazat, ha a jelszó legalább 8 karakter, és mindhárom másik függvény igazat ad rá.

Ellenőrzés: `eros("Alma1234")` → 1, `eros("alma1234")` → 0, `eros("Alma12")` → 0.

7.4. feladat ★★☆☆ - Prímek százig

A 4.7. feladat `prim` függvényével írd ki az összes prímszámot 1 és 100 között, és számold meg őket.

Ellenőrzés: a sor így kezdődik: `2 3 5 7 11`, és összesen 25 prímszám van.

7.5. feladat ★★☆☆ - Caesar-rejtjel

Írj `char eltol(char c, int k)` függvényt, amely egy kisbetűt `k` hellyel eltol az ábécében (a végén körbefordulva). Írj `std::string titkosit(std::string s, int k)` függvényt is, amely az `eltol` segítségével a szó minden betűjét eltolja.

Ellenőrzés: `titkosit("alma", 3)` → `dopd`, `titkosit("xyz", 3)` → `abc`. A visszafejtéshez nem kell új függvény: `titkosit("dopd", 23)` → `alma`.

7.6. feladat ★★☆☆ - Tökéletes számok

Egy szám tökéletes, ha megegyezik a nála kisebb osztóinak összegével ($6 = 1 + 2 + 3$). Írj `int osztokOsszege(int n)` és `bool tokeletes(int n)` függvényt, és keresd meg a tökéletes számokat 1 és 1000 között.

Ellenőrzés: `osztokOsszege(12)` → 16. Három tökéletes számot kell találnod: 6, 28, 496.

Hibakereső: mit jelent a hibaüzenet?

Keresd meg a táblázatban azt a sort, amely hasonlít arra, amit látsz. A hibaüzenet pontos szövege fordítónként kicsit eltérhet.

Amit látsz	Mit jelent?	Mit tegyél?
'vonal' was not declared in this scope	A fordító nem ismeri ezt a nevet.	Ha függvény: a main előtt van megírva? Jól írtad a nevét? Ha változó: egy másik függvényé, add át paraméterként.
too few arguments to function	Kevesebb adatot adtál át, mint ahány paramétere a függvénynek van.	Számold meg a paramétereket a függvény első sorában és a hívásban.
too many arguments to function	Több adatot adtál át, mint ahány paraméter van.	Ugyanaz: a két darabszámnak egyeznie kell.
invalid conversion vagy cannot convert	Más típusú adatot adtál át, mint amelyet a paraméter vár.	Nézd meg a paraméter típusát a függvény első sorában.
void value not ignored as it ought to be	Egy void függvény eredményével próbálsz dolgozni.	A függvény típusa legyen int (vagy más), és legyen benne return .
return-statement with a value, in function returning 'void'	void függvényből próbálsz értéket visszaadni.	Írd át a függvény típusát arra, amit visszaad.
no return statement in function returning non-void	Hiányzik a return (ez csak figyelmeztetés).	Minden lehetséges úton legyen return .
expected ';' vagy expected '}'	Lemaradt egy pontosvessző vagy egy kapcsos zárójel.	Nézd meg a jelzett sort és az előtte lévőket.
Nincs hiba, de nem jelenik meg semmi	A függvényt nem hívtad meg, vagy az eredményét nem írtad ki.	Van hívás a main -ben? Ott van előtte a std::cout ?
Nincs hiba, de értelmetlen szám jelenik meg, vagy a program összeomlik	Valószínűleg hiányzik egy return .	Nézd végig a függvényt: minden úton van return ?
A változó értéke nem változott meg	A függvény csak másolatot kapott.	Lásd az 5. fejezetet: return vagy & .

Összefoglaló táblázat (puska)

Minta	Jelentés
<code>void nev() { ... }</code>	függvény paraméter és eredmény nélkül
<code>void nev(int a, char b) { ... }</code>	függvény két paraméterrel
<code>int nev(int x) { return x * 2; }</code>	függvény, amely egész számot ad vissza
<code>bool nev(int x) { return x > 0; }</code>	függvény, amely igazat vagy hamisat ad vissza
<code>nev(5);</code>	hívás
<code>int e = nev(5);</code>	hívás, az eredmény eltárolásával
<code>return ertekek;</code>	eredmény visszaadása; a függvény azonnal véget ér
<code>void nev(int& x) { ... }</code>	a függvény az eredeti változót módosítja
<code>int nev(std::vector<int> v) { ... }</code>	függvény vektor paraméterrel
<code>std::string nev(std::string s) { ... }</code>	függvény, amely sztringet kap, és sztringet ad vissza

Figyelem! Három dolog, amit soha ne felejts el: a függvényt a `main` elé kell írni; a kiírás és a visszaadás (`return`) nem ugyanaz; a paraméter másolat, hacsak nem írsz a típusa után `&` jelet.